

Trabalho Final de IA e ML - ChefBot

Disciplina: Inteligência Artificial e Machine Learning

Professor: Dr. Tassio Sirqueira

Integrantes: Gabriel Krepker, Gabriel Monteiro, Gustavo Lopes, João Victor da Costa, Rafael Lima Henriques

1. Introdução

1.1 Objetivo

Este trabalho apresenta o desenvolvimento do **ChefBot**, um chatbot temático voltado a culinária que combina técnicas clássicas de Processamento de Linguagem Natural (NLP) com um Modelo de Linguagem de Grande Escala (LLM) em uma arquitetura híbrida. O objetivo é demonstrar como uma base de conhecimento estruturada, consultada antes do LLM, pode aumentar o controle, a previsibilidade e a confiabilidade das respostas, ao mesmo tempo em que o LLM amplia a cobertura do agente para perguntas fora do domínio mapeado.

1.2 Contexto

Chatbots evoluíram de sistemas baseados em regras (ELIZA, 1966) para arquiteturas estatísticas com aprendizado de máquina, e, mais recentemente, para soluções centradas em LLMs (GPT, Llama, Qwen). Cada geração trouxe ganhos em fluência, mas também novos problemas: **alucinações**, custo computacional e perda de controle sobre o que o agente diz. A resposta contemporânea a esses problemas é a **arquitetura híbrida**, em que um componente determinístico (base de conhecimento, recuperação por similaridade) atende às perguntas que pode atender, e o LLM funciona como mecanismo de fallback para o que escapa do escopo conhecido. Esse padrão é hoje a base de sistemas comerciais como assistentes corporativos e agentes de suporte, e está acessível à comunidade graças à plataforma **Hugging Face**, que distribui modelos open-source prontos para uso.

1.3 Justificativa

O estudo e a construção de chatbots híbridos é relevante por três motivos diretamente endereçados neste trabalho:

- **Controle e confiabilidade:** respostas vindas da base de conhecimento são auditáveis, reproduzíveis e livres de alucinação. O ChefBot prioriza esse caminho sempre que possível.
 - **Cobertura via LLM:** nenhuma base finita resolve todas as perguntas de um usuário real. O LLM, como complemento, amplia a utilidade do agente sem exigir manutenção exaustiva de regras.
 - **Escolha do modelo:** utilizamos o **Qwen/Qwen2.5-3B-Instruct** (Alibaba Cloud) em precisão **fp16** (sem quantização). É um modelo *open-weights*, sem *gating* na Hugging Face, com forte desempenho multilíngue (incluindo PT-BR) e custo computacional compatível com hardware doméstico (RTX 4060, 8 GB de VRAM). A escolha do porte 3B (em vez do 7B previsto inicialmente) é detalhada na seção 4.3.
-

2. Fundamentação Teórica

2.1 Definição de Chatbots

Chatbots são agentes conversacionais capazes de interpretar entradas em linguagem natural e produzir respostas coerentes. Suas arquiteturas clássicas são:

- **Baseada em regras:** padrões fixos (ex.: AIML, expressões regulares). Alto controle, baixa flexibilidade.
- **Recuperação (retrieval):** o agente busca a resposta mais semelhante em um conjunto curado, tipicamente via TF-IDF ou embeddings. Boa precisão para perguntas previsíveis.
- **Geração via LLM:** o modelo gera a resposta token a token. Máxima fluência, mas suscetível a alucinações.

O ChefBot combina os três: **regras** para intenções sociais e busca por ingredientes/nome de receita, **retrieval** (TF-IDF) sobre uma base de FAQ culinária, e **LLM** como fallback final.

2.2 Inteligência Artificial em chatbots

Aplicam-se aqui dois grandes blocos:

- **Classificação de intenção** com Naive Bayes - categorizar a mensagem do usuário em uma classe pré-definida (saudação, busca por nome, pergunta culinária, etc.).
- **Recuperação semântica** - encontrar, na base de conhecimento, o item textualmente mais semelhante à pergunta.

2.3 Machine Learning

Utilizamos **aprendizado supervisionado**:

- O classificador de intenção (MultinomialNB do scikit-learn) é treinado sobre pares (frase, classe) em nlp.py.
- O retriever TF-IDF não é supervisionado no sentido tradicional, mas usa a base de FAQ rotulada como ground-truth.

Não houve necessidade de aprendizado por reforço nem não-supervisionado para os objetivos do trabalho.

2.4 Processamento de Linguagem Natural (PLN)

Técnicas aplicadas, todas em nlp.py:

- **Tokenização**: `nltk.word_tokenize` com configuração `language="portuguese"`.
- **Normalização**: remoção de acentos (`unicodedata.NFKD`) e *lower-casing*.
- **Remoção de stopwords**: lista de stopwords PT do NLTK.

- **Lematização:** WordNetLemmatizer aplicado a verbos.
 - **Vetorização:** representação binária bag-of-words para o classificador de intenção; TF-IDF com n-grams (1, 2) para o retriever da FAQ.
 - **Modelos de linguagem (transformers):** o LLM Qwen2.5-3B é um decoder Transformer que gera a resposta no caminho de fallback. Carregado via transformers + accelerate em fp16 na GPU (com fallback para CPU), em [llm.py](#).
-

3. Estado da Arte

3.1 Revisão de literatura (curta)

A literatura recente em chatbots está dominada pelo paradigma **Retrieval-Augmented Generation (RAG)** (Lewis et al., 2020), em que um LLM gera respostas condicionadas a documentos recuperados de uma base externa. Trabalhos contemporâneos como o sistema de suporte da Anthropic e os assistentes corporativos da OpenAI seguem essa linha: um retriever fornece *grounding*, o LLM redige a resposta.

O ChefBot é uma versão **simplificada e didática** desse padrão: em vez de sempre passar pelo LLM com contexto, decidimos primeiro se a base **basta**. Quando basta, o LLM nem é invocado, o que reduz custo, latência e risco de alucinação. Quando não basta, o LLM recebe pergunta + contexto opcional (receitas relacionadas) e responde livremente.

Comparado ao estado da arte:

Aspecto	Estado da arte (RAG comercial)	ChefBot
Recuperação	Embeddings densos (e5, BGE)	TF-IDF (mais leve, suficiente para escopo pequeno)
LLM	GPT-4, Claude, Llama-70B	Qwen2.5-3B em fp16 (open-source, local)

Aspecto	Estado da arte (RAG comercial)	ChefBot
Latência	sub-segundo	~20–25s no caminho LLM, <5ms no caminho base
Custo	API paga por token	Zero após download do modelo
Auditabilidade	Boa	Excelente (caminho base é 100% determinístico)

3.2 Tecnologias utilizadas

- **Python 3.13**
- **NLTK 3.8** - tokenização, stopwords, lematização
- **scikit-learn** - MultinomialNB, TfidfVectorizer
- **Hugging Face Transformers 4.45+** - carregamento do Qwen2.5-3B
- **PyTorch + CUDA 12.4** - inferência em GPU (RTX 4060) com fp16
- **Flask 3** - servidor web, endpoints REST
- **HTML/CSS/JS** - interface de chat (preservada do trabalho anterior)

3.3 Desafios e limitações

- **Alucinação do LLM:** mesmo o Qwen2.5-3B pode inventar receitas ou quantidades. Mitigação: caminho preferencial é a base estruturada, quando o LLM é invocado, o system prompt o restringe ao domínio culinário e a respostas curtas.
- **Dependência de dados:** o desempenho do classificador de intenção e do retriever depende diretamente do tamanho/qualidade do TREINO e do FAQ. Ambos foram construídos manualmente.
- **Custos computacionais:** o Qwen-3B em fp16 consome ~6 GB de VRAM e responde em ~20–25s na RTX 4060 (até 400 tokens novos). Carregamento inicial ~15–45s. Por isso o carregamento é *lazy* + assíncrono e a base é o caminho preferencial.

- **Questões éticas:** o LLM pode reproduzir vieses de pré-treinamento. Em um domínio "seguro" como culinária o risco é menor, mas o system prompt força o agente a recusar perguntas fora de escopo.
-

4. Metodologia

4.1 Justificativas

Tema (culinária / receitas): o tema dá continuidade ao primeiro trabalho da disciplina (também ChefBot), permite uma base de conhecimento estruturada natural (receitas, ingredientes, técnicas), e oferece uma fronteira clara entre o que a base sabe (receitas cadastradas, FAQ) e o que precisa do LLM (perguntas culinárias gerais, substituições atípicas, explicações químicas/biológicas).

Modelo Hugging Face - Qwen2.5-3B-Instruct: escolhido por:

1. **Equilíbrio entre qualidade e latência:** nosso fallback gera respostas culinárias curtas (até ~400 tokens). O Qwen2.5-7B previsto inicialmente em 4-bit não cabia confortavelmente nos 8 GB junto do restante da stack e tinha latência maior; o 3B em fp16 cabe inteiro na GPU e mantém qualidade suficiente no escopo culinário.
2. **Distribuição open-weights sem gating** - diferentemente de Llama-3 e Mistral, qualquer pessoa pode baixar sem aceitar termos adicionais, garantindo reprodutibilidade.
3. **Cabe em fp16 na GPU sem quantização** - ~6 GB de VRAM na RTX 4060 de 8 GB, dispensando a complexidade do bitsandbytes (NF4/QLoRA) que o 7B exigiria.
4. **Treinado com forte presença de português**, mantendo a qualidade multilíngue característica da família Qwen2.5 mesmo no porte menor.
5. **Caminho da base é instantâneo** (<5 ms), cobrindo a maioria das interações; o LLM (mais lento, ~20-25s) só entra quando a base não cobre.

Justificativa de design: preferimos um modelo menor e mais simples (sem quantização) a um modelo maior com complicações de stack (bitsandbytes). Para perguntas factuais curtas restritas a um domínio culinário, modelos de 3B mostraram-se suficientes na nossa avaliação qualitativa.

Extras adotados:

- *Endpoint /stats* expondo métricas em tempo real (taxa de uso da base vs LLM, tempo médio de cada caminho).
- *Preload* assíncrono do modelo no startup do Flask, escondendo a latência de download/carregamento do usuário.
- *Grounding* opcional: quando o LLM é invocado e existem receitas relacionadas aos tokens da pergunta, elas são injetadas no prompt como contexto.
- **Filtro de escopo (guardrail de domínio)**: para evitar que o chatbot responda perguntas fora de culinária, o LLM só é acionado se a mensagem (a) encostar no vocabulário culinário derivado da própria base (base.parece_culinario) **ou** (b) for aprovada por um guardião LLM determinístico SIM/NÃO (llm.dentro_do_escopo). Só bloqueia quando ambos discordam do tema, devolvendo uma recusa educada. O *system prompt* do Agente 2 também reforça a recusa e proíbe inventar fatos.

4.3 Evolução do desenvolvimento (modelos e testes)

O projeto passou por iterações relevantes que valem registro:

Modelos

1. **Trabalho 1 (v1, em `v1\_antigo/`)**: sem LLM - apenas regras + Naive Bayes (NLTK). Servia de base de intenções, mas não respondia perguntas abertas.
2. **Planejamento do Trabalho Final**: previa **Qwen2.5-7B-Instruct** em quantização **4-bit (NF4 / bitsandbytes)**, visando maior qualidade.
3. **Implementação final**: trocamos para **Qwen2.5-3B-Instruct em fp16**. Motivos: o 7B em 4-bit competia por VRAM com o resto da stack nos 8 GB

da RTX 4060 e adicionava a complexidade do bitsandbytes; o 3B em fp16 cabe inteiro na GPU, dispensa quantização e entrega qualidade suficiente no domínio restrito de culinária. O modelo é reusado por **dois agentes** (Agente 1 = classificador de intenção; Agente 2 = gerador de resposta), sem custo extra de VRAM.

4. **Ajustes de geração contra alucinação:** a temperatura do Agente 2 foi reduzida de **0.7 para 0.3** e o *system prompt* passou a proibir invenção de técnicas/medidas, após observarmos alucinações em temas de nicho.

Base de conhecimento e retrieval

- O limiar de similaridade da FAQ (TF-IDF) foi calibrado **0.60 para 0.70 para 0.62** ao longo dos testes.
- A FAQ passou a ser **pré-processada com o tokenizador do NLP** (remoção de stopwords + lematização) antes do TF-IDF, elevando o *recall* de paráfrases (ex.: "*como faço para deixar o arroz soltinho?*" passou de 0.56 para 0.68 de similaridade, voltando a ser respondida pela base).
- As respostas da FAQ foram **reescritas com formatação estruturada** (negrito + listas) e o conteúdo enriquecido/corrigido.
- O módulo da base foi renomeado de kb.py para **`base.py`** para deixar o papel explícito.

Testes

- A suíte testar.py reúne **13 cenários** (conversas completas em sessões isoladas) que exercitam a máquina de estados inteira (sondagem para passo a passo para conclusão) e marcam a **fonte** (base, base-faq, llm, fora-escopo) e o **tempo** de cada turno.
- Modo `--rapido` pula os cenários que dependem do LLM, para validação rápida do caminho determinístico sem GPU.
- Os números reais da última execução estão na seção 5.

4.2 Desenvolvimento do ChefBot

Construção do agente

O agente é composto por quatro módulos independentes:

Módulo	Responsabilidade
<u>nlp.py</u>	Pré-processamento NLTK + classificador de intenção
<u>base.py</u>	Base estruturada: receitas + FAQ + retrieval + filtro de escopo
<u>llm.py</u>	Carregamento e inferência do Qwen2.5-3B (2 agentes)
<u>chatbot.py</u>	Orquestração: aplica o fluxo base para LLM

Lógica de decisão (base para LLM)

O fluxo de decisão do orquestrador funciona da seguinte forma: a mensagem do usuário é processada pela função de detecção de intenção, que retorna uma intenção e um grau de confiança. Com base nesse resultado, o agente segue uma das rotas:

- a) se o Naive Bayes está confiante e a intenção é social (saudação, despedida ou agradecimento), o agente responde com uma mensagem pré-definida da base;
- b) se o Naive Bayes está confiante e a mensagem é uma busca de receita, o agente consulta a base de receitas e apresenta as opções encontradas;
- c) se a pergunta atinge a FAQ com similaridade TF-IDF maior ou igual a 0,62, a resposta vem diretamente da FAQ da base;
- d) se a intenção é ambígua, o Agente 1 (LLM) é acionado apenas para classificar a intenção;
- e) caso contrário, e estando dentro do escopo culinário, o Agente 2 (LLM) gera a resposta livre.

Os contadores em chatbot.ESTATISTICAS registram qual caminho foi tomado.

Integração com a base externa

- A base do ChefBot é **local**, em arquivos Python:
- receitas.py contém o dataset estruturado de receitas (nome, ingredientes, categoria, tempo, porções, dificuldade, instruções).
 - A FAQ vive em base.FAQ como pares (pergunta, resposta).

A integração com a Hugging Face é via API local da biblioteca transformers: o modelo é baixado no primeiro from_pretrained e fica em cache em ~/.cache/huggingface.

Arquitetura do sistema

A arquitetura do sistema é organizada em camadas. O navegador (cliente) comunica-se com o servidor Flask, que expõe os endpoints de conversa e de estatísticas. As mensagens recebidas são encaminhadas ao orquestrador (chatbot.py), responsável pela decisão entre o caminho da base e o caminho do LLM. Esse orquestrador apoia-se em dois blocos: o de recuperação determinística, formado pelos módulos de PLN e da base de conhecimento (NLTK e scikit-learn), e o de geração, no módulo do LLM, que carrega o modelo Qwen2.5-3B em precisão fp16. Os dados das receitas residem em um módulo próprio de dados.

5. Resultados e Discussão

Os números abaixo vêm da execução da suíte testar.py (13 cenários conversacionais, **48 turnos** no total) com o modelo já carregado na GPU (RTX 4060), coletados via chatbot.stats() / GET /stats.

5.1 Taxa de resolução por fonte

Fonte	Turnos	Percentual
Base (regras + FAQ + recusa de escopo)	44	91.7%

Fonte	Turnos	Percentual
LLM (Agente 2, fallback gerativo)	4	8.3%
Total	48	100%

A arquitetura híbrida cumpriu o objetivo central: a **base resolveu ~92%** das interações, e o LLM foi acionado apenas para perguntas culinárias **fora** da base (ex.: *orchata*, maturação a seco, sushi). Isso confirma o princípio "base antes do LLM" exigido pelo enunciado.

5.2 Latência por caminho

Caminho	Latência típica	Observação
Base (regra/handler)	< 1 ms	praticamente instantâneo
Base (FAQ TF-IDF)	1–3 ms	vetorização + similaridade
Recusa de escopo	~300 ms	inclui o guardião LLM SIM/NÃO
LLM (Agente 2)	~20–25 s	fp16, até 400 tokens novos na RTX 4060

5.3 Avaliação qualitativa

- **Caminho da base:** respostas **corretas e auditáveis** por construção (conteúdo curado). Após a reescrita, a FAQ entrega respostas formatadas (negrito + listas) com a mesma legibilidade das respostas do LLM.
- **Caminho do LLM:** fluente e bem formatado, mas **suscetível a alucinação** em temas de nicho fora da base (ex.: inventou queijo brie em "frango caipira" e descreveu uma "régua de extração" para ordenhar uma vaca). A temperatura 0.3 e o *system prompt* anti-invenção reduzem, mas não eliminam, daí a estratégia de **ampliar a base** para diminuir a dependência do LLM.
- **Filtro de escopo:** perguntas claramente fora do domínio (ex.: "*quem pintou a mona lisa?*") foram corretamente recusadas como fora-escopo, sem acionar a geração longa.

5.4 Anomalia encontrada nos testes

A suíte revelou um bug de baixa gravidade (pré-existente): a mensagem "tchau" ocasionalmente é roteada para uma busca de receita ("Chá de Camomila") em vez de despedida. Causa: o Naive Bayes classifica despedida mas com baixa confiança, caindo no `buscar_por_nome`, que casa "**cha**" como subcadeia de "**tchau**". É um ponto a corrigir (casamento de nome mais estrito / reforço do treino de despedida).

7. Conclusões e Trabalhos Futuros

O ChefBot demonstrou, na prática, que uma **arquitetura híbrida bem calibrada** entrega o melhor dos dois mundos: a confiabilidade de uma base determinística com a cobertura aberta de um LLM. Principais aprendizados:

- **Arquitetura híbrida (base + LLM):** a separação explícita entre caminhos determinístico e generativo se mostrou eficaz - **~92% das interações** foram resolvidas pela base (instantânea e auditável), sem acionar o LLM. Conter o LLM ao que realmente escapa do escopo reduziu custo, latência e alucinação no domínio coberto.
- **Controle de respostas:** o limiar de similaridade do TF-IDF (calibrado para **0.62**), o **pré-processamento da FAQ com NLTK** (stopwords + lematização) e o **filtro de escopo** foram os mecanismos centrais de controle. Juntos, eles decidem quando a base "basta" e impedem que o agente responda fora de culinária ou force uma resposta ruim da base.
- **Dependência do modelo:** o Qwen2.5-3B em fp16 manteve conversas em PT-BR sobre culinária com qualidade aceitável, mas a **latência (~20–25 s por resposta)** e a **alucinação em temas de nicho** são limitações reais - reforçando que o caminho referencial deve ser sempre a base.

Trabalhos futuros: trocar o TF-IDF por **embeddings densos** (*sentence-transformers* PT) para recall ainda melhor; **expandir a base** de FAQ e receitas para reduzir a fração que cai

no LLM; **streaming de tokens** para mascarar a latência do LLM; e endurecer o roteamento de intenções curtas.

8. Referências

1. Devlin et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.
 2. Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.
 3. Vaswani et al. (2017). *Attention Is All You Need*. NeurIPS.
 4. Bird, S.; Klein, E.; Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly. (Documentação base do NLTK.)
 5. Pedregosa et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR 12.
 6. Qwen Team, Alibaba Cloud (2024). *Qwen2.5 Technical Report*. <https://huggingface.co/Qwen/Qwen2.5-3B-Instruct>
 7. Hugging Face Transformers. <https://huggingface.co/docs/transformers>
-

9. Apêndices

- **Código-fonte:** <https://github.com/Cozta1/Cozta1-ChatBotReceitas> (branch main - versão híbrida base + LLM)
- **Vídeo de demonstração:** *(a gravar antes da apresentação final)*